《加密解密、系统集成与发布》完整课堂讲义

目录

《加密解密、系统集成与发布》完整课堂讲义

- 目录
- 1. 为什么备份软件需要加密
- 2. 密码学核心概念扫盲
  - 2.1 分组密码与流密码
  - 2.2 什么是CBC模式（密码分组链接）
  - 2.3 数据补齐（Padding）
- 3. 从用户密码到加密密钥：密钥派生函数
  - 3.1 问题
  - 3.2 解决方案：哈希函数 + 盐值
  - 3.3 为什么需要两次哈希（`hash1` 和 `hash2`）
- 4. 加密文件格式设计
  - 4.1 公共头 `commonHeaderStruct`
  - 4.2 加密扩展头 `encryptionHeaderStruct`
  - 4.3 加密文件的完整布局
- 5. AES加密解密完整流程（CAES类详解）
  - 5.1 类结构
  - 5.2 加密流程（`encrypt` 函数）
  - 5.3 解密流程（`decrypt` 函数）
  - 5.4 密码验证（`decryptionCheck` 函数）
- 6. DES加密解密实现（CDES类详解）
  - 6.1 DES与AES的对比
  - 6.2 代码差异详解
  - 6.3 使用建议
- 7. 封装成库与ABI兼容性
  - 7.1 为什么要封装成库
  - 7.2 什么是 ABI
  - 7.3 C++ 的 ABI 问题
  - 7.4 为什么用 C 接口暴露 C++ 库
- 8. 动态链接的原理与优势
  - 8.1 静态链接 vs 动态链接
  - 8.2 动态链接的内存共享机制
  - 8.3 在本项目中的应用
- 9. 系统集成：各模块的协作顺序
  - 9.1 完整的数据备份流程
  - 9.2 为什么先压缩后加密
  - 9.3 完整的数据恢复流程
- 10. 需求变更：备份验证功能
  - 10.1 需求背景
  - 10.2 最简单的实现方案：CRC
  - 10.3 代码中的 CRC64 实现
  - 10.4 CRC 的局限性
- 11. 课堂思考题
  - 思考题1
  - 思考题2
  - 思考题3
  - 思考题4

思考题5
思考题6
12. 总结
核心知识点
代码设计亮点
扩展方向

1. 为什么备份软件需要加密

在之前的课程中，我们已经实现了：

- 文件遍历与过滤（`CFileEnumerator`）
- 打包解包（`CMyPack`）：将目录树合并为一个大文件
- 压缩解压（`CHuffman` / `CLZ77`）：减小备份文件的体积

但此时备份文件存在一个严重的安全隐患：**任何能够访问该文件的人，都可以直接解包还原出所有原始数据。**

试想以下场景：

- 备份文件存储在移动硬盘或U盘中，设备丢失
- 备份文件上传到云端存储服务
- 备份文件存放在共享服务器上，被非授权用户访问

加密的目的就是：**即使备份文件落入他人之手，没有正确的密码也无法还原任何数据。**

加密算法分为两大类：

类型	特点	典型算法	在本项目中的用途
对称加密	加密和解密使用**相同**的密钥	DES、3DES、AES	**核心功能**：加密备份文件数据
非对称加密	加密和解密使用**不同**的密钥（公钥/私钥）	RSA、ECC	本课程不涉及
散列算法	将任意长度数据映射为固定长度"指纹"，**不可逆**	MD5、SHA-1、SHA-256	**辅助功能**：密码验证、完整性校验

本项目采用**对称加密**，因为：

- 备份场景中只有一个用户（或一组共享密码的用户）需要访问数据
- 对称加密速度远快于非对称加密，适合加密大量数据
- 实现相对简单

2. 密码学核心概念扫盲

本讲义不涉及复杂的数学原理（如S盒、置换表、椭圆曲线等），仅解释在代码中直接出现的关键概念。

2.1 分组密码与流密码

流密码（Stream Cipher）：

- 逐字节或逐比特加密
- 每个位置的明文用不同的密钥流加密
- 优点：无需补齐，实时性强
- 缺点：如果密钥流重用，容易被攻击

分组密码（Block Cipher）：

- 将明文分成固定大小的块，逐块加密
- DES：每块 **8字节（64位）**
- AES：每块 **16字节（128位）**
- 本项目使用的 **DES-EDE-CBC** 和 **AES-256-CBC** 都属于分组密码

2.2 什么是CBC模式（密码分组链接）

分组密码有多种工作模式，CBC（Cipher Block Chaining）是最常用的一种。

核心思想：每个明文块在加密前，先与前一个密文块进行**异或（XOR）运算**，然后再加密。

```
加密过程：
  密文块0 = 加密(明文块0 ⊕ 初始化向量IV)
  密文块1 = 加密(明文块1 ⊕ 密文块0)
  密文块2 = 加密(明文块2 ⊕ 密文块1)
  ...

解密过程：
  明文块0 = 解密(密文块0) ⊕ IV
  明文块1 = 解密(密文块1) ⊕ 密文块0
  明文块2 = 解密(密文块2) ⊕ 密文块1
  ...
```

CBC模式的两个关键点：

1. **初始化向量（IV）**：第一个块没有"前一个密文块"，因此需要一个随机的IV。即使两份完全相同的明文使用相同密钥加密，只要IV不同，密文就完全不同。
2. **错误传播**：如果某个密文块损坏，会导致两个明文块出错（当前块和下一块）。但在备份场景中，我们更关注数据完整性（CRC64校验）而非实时传输，这是可以接受的。

2.3 数据补齐（Padding）

分组密码要求输入数据长度必须是块大小的整数倍。

- DES：块大小 = 8字节，数据长度必须是8的倍数
- AES：块大小 = 16字节，数据长度必须是16的倍数

PKCS#7补齐规则：

情况	补齐方式
最后一块缺 N 个字节（1 ≤ N ≤ 块大小）	补 N 个字节，每个字节的值都是 N
最后一块恰好满块大小	补 **一整块**，每个字节的值都是块大小（DES补8个0x08，AES补16个0x10）

示例（AES，块大小16）：

- 最后一块只有 10 字节 → 缺 6 字节 → 补 6 个 0x06
- 最后一块刚好 16 字节 → 补 16 个 0x10

好消息：本项目使用的 OpenSSL EVP API **自动处理补齐**，加密时调用 `EVP_EncryptFinal_ex` 自动补块，解密时调用 `EVP_DecryptFinal_ex` 自动移除填充。我们不需要手动实现补齐逻辑。

3. 从用户密码到加密密钥：密钥派生函数

3.1 问题

用户输入的密码可能是任意长度和内容（如 `"123456"`、`"my_secure_password"`、`"密码123"`），但加密算法对密钥有严格的长度要求：

算法	密钥长度（字节）	密钥长度（比特）
DES	8	64
3DES（DES-EDE）	16	128
AES-128	16	128
AES-192	24	192
AES-256	32	256

需要一种方法将用户输入的任意长度密码转换为固定长度的二进制密钥。

3.2 解决方案：哈希函数 + 盐值

代码中实现了 `saltedSHA256` 函数：

```
std::string CDES::saltedSHA256(const std::string input) {
    // 1. 在输入字符串后追加固定盐值
    std::string salted_string = input + HASH_SALT;

    // 2. 使用 SHA-256 计算哈希值
    //    SHA-256 输出固定 32 字节（256位）的二进制数据
    unsigned char md_value[EVP_MAX_MD_SIZE];
    EVP_DigestInit_ex(mdctx, md, NULL);
    EVP_DigestUpdate(mdctx, salted_string.c_str(), salted_string.length());
    EVP_DigestFinal_ex(mdctx, md_value, &md_len);

```

```
    // 3. 将 32 字节的二进制数据转换为 64 字符的十六进制字符串
    char hexstr[md_len * 2 + 1];
    for (unsigned int i = 0; i < md_len; i++) {
        sprintf(ptr, "%02x", md_value[i]);
    }
    return std::string(hexstr);  // 返回 64 字符的字符串
}
```

为什么使用盐值（Salt）？

`HASH_SALT` 定义为 `"databackup-uestc"`。盐值的作用是**防止彩虹表攻击。**

- 攻击者可能预先计算常用密码的哈希值（彩虹表），用于快速破解。
- 加盐后，即使两个用户使用相同密码，哈希结果也完全不同（因为盐值可能不同，这里盐值固定但已足够防止通用彩虹表）。
- 盐值不应该是秘密的，它可以明文存储在代码中。它的作用是增加攻击者的计算成本。

验证加盐的效果：

```
不加盐: SHA256("123456") = "8d969eef6ecad3c29a3a629280e686cf0c3f5d5a86aff3ca12020c923adc6c92"
加盐后: SHA256("123456" + "databackup-uestc") = 完全不同的一串字符
```

3.3 为什么需要两次哈希（`hash1` 和 `hash2`）

代码中执行了两次哈希：

```
std::string hash1 = this->saltedSHA256(key);   // 第一次
std::string hash2 = this->saltedSHA256(hash1); // 第二次
```

两次哈希的角色分工：

变量	来源	用途	长度
`hash1`	对用户密码加盐做 SHA256	**派生加密密钥和IV**	64个十六进制字符（= 32字节二进制）
`hash2`	对 `hash1` 再做一次 SHA256	**密码验证**（存储于文件头）	64个十六进制字符

为什么不直接用 `hash1` 同时做密钥和验证?

如果直接把 `hash1` 存入文件头用于验证密码，攻击者可以读取文件头的 `hash1`，虽然无法逆推出原始密码，但 `hash1` 本身就可以作为密钥使用（因为加密密钥就是从 `hash1` 切出来的）。这意味着攻击者不需要知道原始密码，只要拿到文件头里的 `hash1` 就能解密数据。

使用 `hash2` 作为验证凭证，而加密密钥来自 `hash1`，两者分开，攻击者即使读取了文件头的 `hash2`，也无法推算出 `hash1` 和加密密钥。

4. 加密文件格式设计

4.1 公共头 `commonHeaderStruct`

所有备份文件都包含此结构，由打包模块和压缩模块共用：

```
struct commonHeaderStruct {
    uint64_t CRC64;      // 文件完整性校验值
    uint32_t offset;     // 数据起始偏移量
    uint32_t Algorithm;  // 算法标识（DES_ALGORITHM / AES_ALGORITHM / HUFFMAN_ALGORITHM
等）
};
```

4.2 加密扩展头 `encryptionHeaderStruct`

加密模块在此结构基础上扩展：

```
struct encryptionHeaderStruct {
    commonHeaderStruct common;                 // 公共头
    char key[FILE_ENCRYPO_KEY_LENGTH];        // hash2 的副本（用于密码验证）
};
```

其中 `FILE_ENCRYPO_KEY_LENGTH` 是固定值（64字节），对应 `hash2` 的十六进制字符串长度。

4.3 加密文件的完整布局

```
┌──────────────────────────────────────────────────────────────┐
| encryptionHeaderStruct (固定长度)                             |
|   ├── commonHeaderStruct                                     |
|   |    ├── CRC64 = 文件校验值                                 |
|   |    ├── offset = sizeof(encryptionHeaderStruct)           |
|   |    └── Algorithm = AES_ALGORITHM 或 DES_ALGORITHM        |
|   └── key[64] = hash2 的十六进制字符串                        |
├──────────────────────────────────────────────────────────────┤
| 密文数据区 (从 offset 处开始)                                  |
|   EVP_EncryptUpdate 输出的所有密文块 + EVP_EncryptFinal_ex    |
|   补齐后的最后一整块                                          |
└──────────────────────────────────────────────────────────────┘
```

5. AES加密解密完整流程（CAES类详解）

5.1 类结构

```
class CAES : public IEncryptionAlgorithm {
public:
    int encrypt(std::string file, std::string targetFile, std::string key) override;
    int decrypt(std::string file, std::string targetFile, std::string key) override;
    std::string getAlgorithmName() override;

```

```
protected:
    int encryptionCheck(...) override;
    int decryptionCheck(...) override;
    std::string saltedSHA256(const std::string input);  // 密钥派生函数
};
```

5.2 加密流程（`encrypt` 函数）

步骤1：参数校验

```
if (encryptionCheck(file, targetFile, key) < 0) {
    return -1;
}
```

检查：

- 源文件是否存在且可读
- 源文件是否为普通文件（不加密设备文件、管道等）

步骤2：密钥派生

```
std::string hash1 = this->saltedSHA256(key);   // 64字符十六进制
std::string hash2 = this->saltedSHA256(hash1); // 64字符十六进制

// 从 hash1 中切出 AES-256 需要的材料
std::string aes_key = hash1.substr(AES_KEY_POS, AES_KEY_LENGTH);  // 前32字符 → 密钥
std::string aes_iv  = hash1.substr(AES_IV_POS, AES_IV_LENGTH);    // 第33-48字符 → IV
```

宏定义说明：

```
#define AES_KEY_POS 0        // 从第0个字符开始
#define AES_KEY_LENGTH 32    // 取32个字符（对应256位密钥）
#define AES_IV_POS 32        // 从第32个字符开始
#define AES_IV_LENGTH 16     // 取16个字符（对应128位IV）
```

步骤3：写入加密头

```
commonHeaderStruct chs = {
    .CRC64 = 0,  // 稍后通过 updateFileCRC64 回填
    .offset = sizeof(encryptionHeaderStruct),
    .Algorithm = AES_ALGORITHM
};

encryptionHeaderStruct ehs = { .common = chs };
memset(ehs.key, 0, FILE_ENCRYPO_KEY_LENGTH);
memcpy(ehs.key, hash2.c_str(), hash2.length());  // 存储 hash2 用于验密

sfTarget.writeN(&ehs, sizeof(ehs));
```

步骤4：初始化解密上下文

```
EVP_CIPHER_CTX *ctx = EVP_CIPHER_CTX_new();  // 创建上下文对象
EVP_CIPHER *cipher = EVP_CIPHER_fetch(NULL, "AES-256-CBC", "provider=default");
EVP_EncryptInit_ex(ctx, cipher, NULL,
                   (const unsigned char*)aes_key.c_str(),
                   (const unsigned char*)aes_iv.c_str());
```

`EVP_CIPHER_fetch` 的第三个参数 `"provider=default"` 是 OpenSSL 3.0 引入的提供者机制，指定使用默认的加密算法提供者。如果不指定，某些系统可能找不到算法实现。

`EVP_EncryptInit_ex` 的第四个和第五个参数分别是密钥和IV的指针。对于 AES-256-CBC：

- 密钥必须是 32 字节
- IV 必须是 16 字节

步骤5：分块加密并写入

```
unsigned char inputBuffer[1024];   // 明文缓冲区（1KB）
unsigned char outputBuffer[1024 + EVP_MAX_BLOCK_LENGTH];  // 密文缓冲区（略大）

int bytesRead = 0, bytesWritten = 0;

while ((bytesRead = sfFile.readN(inputBuffer, sizeof(inputBuffer))) > 0) {
    EVP_EncryptUpdate(ctx, outputBuffer, &bytesWritten,
                      inputBuffer, bytesRead);
    sfTarget.writeN(outputBuffer, bytesWritten);
}

// 处理最后不足一个块的数据（自动补齐）
EVP_EncryptFinal_ex(ctx, outputBuffer, &bytesWritten);
sfTarget.writeN(outputBuffer, bytesWritten);
```

`EVP_EncryptUpdate` 的输入输出关系：

- **输入**：明文数据（可能不足一个块，也可能包含多个块）
- **输出**：加密后的密文数据（长度可能与输入相同，也可能稍长）
- `bytesWritten` 是实际输出的密文字节数

`EVP_EncryptFinal_ex` 的作用：

- 处理缓冲区中残留的数据
- 应用 PKCS#7 补齐（如果最后一块不满块大小）
- 输出最后一块密文

步骤6：清理资源

```
EVP_CIPHER_free(cipher);
EVP_CIPHER_CTX_free(ctx);
updateFileCRC64(targetFile);  // 回填 CRC64 校验值
```

5.3 解密流程（`decrypt` 函数）

步骤1：校验与密钥派生

```
if (decryptionCheck(file, targetFile, key) < 0) return -1;

std::string hash1 = this->saltedSHA256(key);
std::string aes_key = hash1.substr(AES_KEY_POS, AES_KEY_LENGTH);
std::string aes_iv  = hash1.substr(AES_IV_POS, AES_IV_LENGTH);
```

步骤2：跳过文件头

```
Utils::CSimpleFile sfFile(file, O_RDONLY, -1);
// 跳过 encryptionHeaderStruct，直接定位到密文数据起始处
if (sfFile.seekFile(sizeof(encryptionHeaderStruct), SEEK_SET) < 0) {
    return -1;
}
```

步骤3：初始化解密上下文

```
EVP_CIPHER_CTX *ctx = EVP_CIPHER_CTX_new();
EVP_CIPHER *cipher = EVP_CIPHER_fetch(NULL, "AES-256-CBC", "provider=default");
EVP_DecryptInit_ex(ctx, cipher, NULL,
                   (const unsigned char*)aes_key.c_str(),
                   (const unsigned char*)aes_iv.c_str());
```

步骤4：分块解密并写入

```
while ((bytesRead = sfFile.readN(inputBuffer, sizeof(inputBuffer))) > 0) {
    EVP_DecryptUpdate(ctx, outputBuffer, &bytesWritten,
                      inputBuffer, bytesRead);
    sfTarget.writeN(outputBuffer, bytesWritten);
}

// 处理最后一块（自动移除 PKCS#7 填充）
EVP_DecryptFinal_ex(ctx, outputBuffer, &bytesWritten);
sfTarget.writeN(outputBuffer, bytesWritten);
```

`EVP_DecryptFinal_ex` 的作用：

- 处理最后一块密文
- 自动验证并移除 PKCS#7 填充
- 如果填充格式不正确，函数会返回错误（表示数据可能被篡改或密钥错误）

5.4 密码验证（`decryptionCheck` 函数）

```cpp
int CAES::decryptionCheck(std::string file, std::string targetFile, std::string key) {
    // 1. 检查文件是否存在和可读
    // 2. 检查文件类型是否普通文件
    // 3. CRC64 校验（检查文件是否完整）
    // 4. 读取 encryptionHeaderStruct
    encryptionHeaderStruct ehs;
    sf.readN(&ehs, sizeof(ehs));

    // 5. 校验算法标识和偏移量
    if (ehs.common.Algorithm != AES_ALGORITHM ||
        ehs.common.offset != sizeof(encryptionHeaderStruct)) {
        return BACKUP_DATA_FILE_ERROR;
    }

    // 6. 计算用户输入密码的 hash2
    std::string hash1 = this->saltedSHA256(key);
    std::string hash2 = this->saltedSHA256(hash1);

    // 7. 比对 hash2 是否与文件头中存储的一致
    if (hash2 != std::string(ehs.key, FILE_ENCRYPO_KEY_LENGTH)) {
        return ENCRYPT_KEY_ERROR;  // 密码错误
    }
    return 0;
}
```

6. DES加密解密实现（CDES类详解）

6.1 DES与AES的对比

对比项	DES（CDES）	AES（CAES）
算法名称	`DES-EDE-CBC`（三重DES）	`AES-256-CBC`
块大小	8 字节	16 字节
密钥长度	16 字节（从 hash1 取前16字符）	32 字节（从 hash1 取前32字符）
IV	未指定（传 NULL，OpenSSL 使用全0 IV）	16 字节（从 hash1 取第33-48字符）
安全性	较低（已过时，但比不加密强）	较高（现代标准）
性能	较慢（三重DES需要三次加密）	较快（硬件加速支持广泛）

6.2 代码差异详解

密钥派生差异：

```
// CDES::encrypt
std::string hash1 = this->saltedSHA256(key);
std::string hash2 = this->saltedSHA256(hash1);
std::string des_key = hash1.substr(DES_KEY_POS, DES_KEY_LENGTH);  // 16字符
// 注意：DES 不显式使用 IV，EVP_EncryptInit_ex 第四个参数传 NULL

// CAES::encrypt
std::string hash1 = this->saltedSHA256(key);
std::string hash2 = this->saltedSHA256(hash1);
std::string aes_key = hash1.substr(AES_KEY_POS, AES_KEY_LENGTH);  // 32字符
std::string aes_iv  = hash1.substr(AES_IV_POS, AES_IV_LENGTH);    // 16字符
```

为什么 DES 没有显式指定 IV?

在 `EVP_EncryptInit_ex` 中，如果 IV 参数传 `NULL`，OpenSSL 会使用全零 IV。在 CBC 模式下，全零 IV 本身并不安全（相同明文的第一个块会产生相同密文），但由于 DES 在本项目中已不推荐使用，代码保留了这种简化写法。**强烈建议学生使用 AES 并显式指定 IV。**

6.3 使用建议

在课堂演示时，可以这样向学生解释选择：

> “DES 的历史可以追溯到 1970 年代，它的 56 位密钥长度在今天可以用暴力破解（2016 年有研究团队在 20 小时内破解了 DES 密钥）。3DES（DES-EDE）通过三次加密提高了安全性，但性能较差。AES 是 2001 年确定的标准，256 位密钥在可预见的未来无法被暴力破解。因此，在备份软件中，**请优先选择 AES。**”

7. 封装成库与ABI兼容性

7.1 为什么要封装成库

在大型软件工程中，**核心逻辑与界面实现分离**是重要的设计原则：

- **界面层**：可以使用 Qt、MFC、.NET 等技术，专注于用户体验
- **核心层**：使用 C/C++ 编写，专注于算法性能和稳定性
- **接口层**：定义清晰的 API，使得界面层可以调用核心层的功能

这样做的好处：

1. **技术栈独立**：前端和后端团队可以各自选择最适合的技术
2. **编译隔离**：修改界面代码不需要重新编译核心库
3. **单元测试**：核心库可以独立进行测试
4. **复用性**：同一个核心库可以被不同的界面程序使用（命令行、GUI、Web服务）

7.2 什么是 ABI

ABI（Application Binary Interface，应用二进制接口） 是指不同程序模块在**二进制层面**的对接规范。

ABI 包含的内容：

- 函数调用约定（参数如何传递、返回值如何处理）
- 数据类型的大小和内存布局

- 寄存器使用规则
- 堆栈结构
- 名称修饰规则（Name Mangling）
- 异常处理机制
- 模块初始化和终止过程

不同平台有不同的 ABI 标准：

- Linux：System V AMD64 ABI
- Windows：Microsoft x64 ABI
- 不同架构（x86、ARM、RISC-V）各有自己的 ABI

7.3 C++ 的 ABI 问题

C++ 标准**没有规定 ABI**，这意味着：

- GCC 编译的 C++ 库与 Clang 编译的 C++ 程序可能二进制不兼容
- 不同版本的 GCC 生成的 C++ 代码可能不兼容
- 不同操作系统上的 C++ ABI 差异更大

导致 C++ ABI 不兼容的主要原因：

名称修饰（Name Mangling）：
C++ 支持函数重载，编译器需要将函数名编码为唯一的符号名。

```
// 源代码
class CAES {
    int encrypt(std::string file, std::string key);
    int encrypt(std::string file, std::string key, int mode);
};

// GCC 可能修饰为：
// _ZN4CAES7encryptESsSs   (第一个)
// _ZN4CAES7encryptESsSsi  (第二个)

// Clang 可能使用不同的修饰规则
```

标准库差异：

- `std::string` 的内存布局在不同编译器间可能不同
- 异常处理的实现机制可能不同

7.4 为什么用 C 接口暴露 C++ 库

C 语言的 ABI 更稳定、更通用：

特性	C++	C
名称修饰	有（依赖于编译器）	无（函数名即为符号名）

特性	C++	C
函数重载	支持（导致复杂修饰）	不支持（简单直接）
异常处理	复杂（不同编译器实现不同）	无（用返回值传递错误）
标准库	复杂（std::string 等）	简单（char* 等）
跨语言调用	困难	几乎所有语言都支持 C 接口

使用 C 接口的推荐方式：

```
// 头文件（C++ 代码中，供外部调用）
#ifdef __cplusplus
extern "C" {
#endif

// 返回句柄（不透明指针），封装 C++ 对象
void* create_encryptor(void);
void destroy_encryptor(void* handle);
int encrypt_data(void* handle, const char* input, char* output, const char* key);

#ifdef __cplusplus
}
#endif
```

```
// 实现（.cpp 文件）
extern "C" {
    void* create_encryptor(void) {
        return new CAES();  // 返回 C++ 对象指针，但外部作为 void* 使用
    }

    int encrypt_data(void* handle, const char* input, char* output, const char* key) {
        CAES* obj = static_cast<CAES*>(handle);  // 强制转回 C++ 类型
        return obj->encrypt(input, output, key);
    }
}
```

这样，外部调用者（无论是 C、Python、C# 还是其他语言）只需要包含头文件，链接到编译好的库，就可以使用加密功能。

8. 动态链接的原理与优势

8.1 静态链接 vs 动态链接

静态链接：

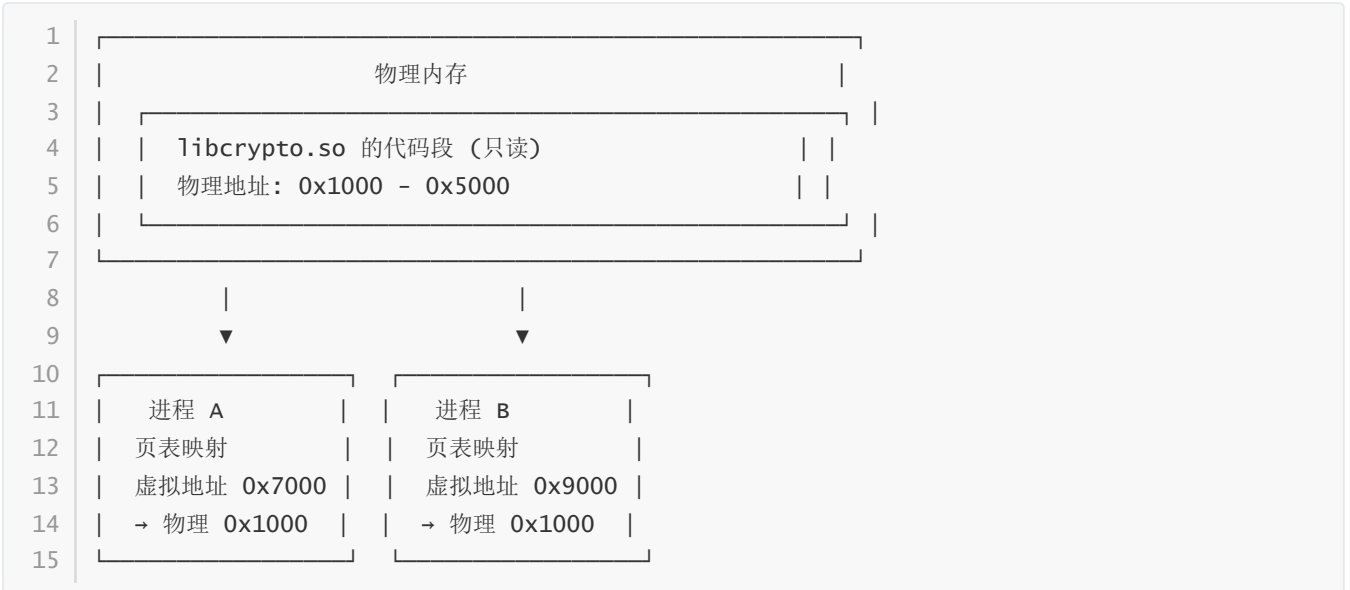
- 在编译时，所有库代码被复制到可执行文件中
- 优点：部署简单，不需要额外文件
- 缺点：文件体积大；库更新时需要重新链接整个程序

动态链接：

- 在运行时，动态加载共享库（`.so` 文件 / `.dll` 文件）
- 优点：节省磁盘和内存空间；库可以独立更新
- 缺点：部署时需要确保库文件存在；存在版本兼容性问题

8.2 动态链接的内存共享机制

在 Linux 系统中，动态链接库通过**页表映射**实现内存共享：



- 库的代码段在物理内存中**只保留一份**副本
- 每个进程通过页表将虚拟地址映射到同一物理地址
- 多个进程可以共享同一个动态库，节省大量内存

8.3 在本项目中的应用

备份软件可以拆分为：

- **核心库**（编译为 `.so` 或 `.dll`）：包含打包、压缩、加密算法
- **界面程序**（可执行文件）：调用核心库功能

这样设计的好处：

1. 修复加密算法的 Bug 时，只需更新 `.so` 文件，不需要重新编译整个程序
2. 可以开发多个不同的界面（命令行、Qt GUI、Web服务），共用同一个核心库
3. 核心库的单元测试可以独立运行

9. 系统集成：各模块的协作顺序

9.1 完整的数据备份流程

```
用户操作：选择源目录 → 设置过滤条件 → 输入密码 → 点击"开始备份"

程序执行：
┌──────────────────────────────────────────────────────────┐
| 1. CFileEnumerator::enumerateFiles                       |
|    遍历源目录，应用过滤条件，生成文件列表                  |
|    输出：vector<string> files                            |
├──────────────────────────────────────────────────────────┤
| 2. CMyPack::pack                                         |
|    将所有文件打包为单一文件 data.dbkp                     |
|    输出：data.dbkp（可能包含目录、链接、设备文件等）       |
├──────────────────────────────────────────────────────────┤
| 3. CHuffman 或 CLZ77::compress                           |
|    对打包文件进行无损压缩                                 |
|    输出：data.dbkp.huff（体积显著缩小）                   |
├──────────────────────────────────────────────────────────┤
| 4. CAES 或 CDES::encrypt                                 |
|    使用用户密码加密压缩后的文件                           |
|    输出：data.dbkp.huff.aes（最终备份文件）               |
└──────────────────────────────────────────────────────────┘
```

9.2 为什么先压缩后加密

这是一个重要的工程决策。**必须压缩后再加密，顺序不能颠倒。**

原因：

压缩算法（如 LZ77 和 Huffman）依赖数据中的**冗余**来实现压缩：

- 重复的字符串（LZ77）
- 不均匀的字符频率（Huffman）

加密算法的输出是**伪随机数据**：

- 每个字节出现的概率近似相等（无法用 Huffman 压缩）
- 几乎没有重复模式（无法用 LZ77 压缩）

如果将加密后的数据再压缩，压缩率将非常低（接近 1:1），浪费了计算资源。

正确顺序：

```
明文（有冗余）→ 压缩 → 密文（无冗余）→ 加密
```

错误顺序：

```
明文（有冗余）→ 加密 → 密文（无冗余）→ 压缩（无效）
```

9.3 完整的数据恢复流程

```
用户操作: 选择备份文件 → 输入密码 → 点击"开始还原"

程序执行:
┌─────────────────────────────────────────────────────────┐
| 1. CAES 或 CDES::decrypt                                |
|    使用用户密码解密备份文件                              |
|    校验密码正确性 (通过文件头中的 hash2)                 |
|    输出: data.dbkp.huff (解密后的压缩文件)               |
├─────────────────────────────────────────────────────────┤
| 2. CHuffman 或 CLZ77::decompress                        |
|    解压文件                                             |
|    输出: data.dbkp (打包文件)                           |
├─────────────────────────────────────────────────────────┤
| 3. CMyPack::unpack                                      |
|    将打包文件解包到目标目录                             |
|    恢复目录结构、文件类型、元数据                       |
|    输出: 完整的原始目录树                               |
└─────────────────────────────────────────────────────────┘
```

10. 需求变更：备份验证功能

10.1 需求背景

在实际使用中，用户需要确认备份文件是否完整可用。可能出现的问题：

- 备份过程中系统崩溃，文件不完整
- 存储介质损坏，部分数据丢失
- 网络传输中断，文件损坏

需求：在备份软件中增加验证功能，检查所备份数据的正确性。

10.2 最简单的实现方案：CRC

CRC（Cyclic Redundancy Check，循环冗余校验） 是一种常用的错误检测码。

在备份场景中的用法：

1. 打包/压缩/加密完成后，计算整个文件的 CRC64 值
2. 将 CRC64 值存入文件头（`commonHeaderStruct.CRC64`）
3. 验证时重新计算文件的 CRC64，与存储的值比对
4. 一致则表示文件完整，不一致则表示文件已损坏

10.3 代码中的 CRC64 实现

```
// 在 encrypt 函数末尾
if (updateFileCRC64(targetFile) < 0) {
    std::cout << "更新文件CRC64时失败: " << targetFile << std::endl;
    return -1;
}

// 在 decryptionCheck 函数中
if (checkFileCRC64(file) < 0) {
    std::cout << "文件CRC64校验失败: " << file << std::endl;
    return BACKUP_DATA_FILE_ERROR;
}
```

`updateFileCRC64` 的工作原理：

1. 读取目标文件的所有数据
2. 计算 CRC64 值
3. 将值写回到文件头的 `CRC64` 字段

`checkFileCRC64` 的工作原理：

1. 读取文件头的 `CRC64` 存储值
2. 计算整个文件（除 CRC64 字段自身）的 CRC64 值
3. 比对两者是否一致

10.4 CRC 的局限性

需要向学生说明 CRC 的适用范围：

- CRC 能检测**意外错误**（传输错误、存储介质损坏）
- CRC **不能防止恶意篡改**（攻击者可以同时修改数据和 CRC 值）
- 如果需要防篡改，应使用**数字签名**（如 RSA + SHA256）

在备份场景中，CRC 足以应对磁盘损坏、文件系统错误等常见问题。

11. 课堂思考题

思考题1

问题：为什么代码中要保存 `hash2` 到文件头用于验证密码，而不是直接保存 `hash1`？

参考回答：如果保存 `hash1`，攻击者即使不知道原始密码，拿到文件头中的 `hash1` 后，因为加密密钥就是从 `hash1` 截取的（`hash1.substr(0, 32)`），可以直接用 `hash1` 解密数据。保存 `hash2`（`hash1` 的哈希）做验证，攻击者无法从 `hash2` 反推出 `hash1`，更无法反推出原始密码，安全性更高。

思考题2

问题：如果用户输入的密码是 `"123456"`，加密后的文件头中 `key` 字段的值是什么格式？包含数字和字母吗？

参考回答：`key` 字段存储的是 `hash2`，即经过两次 SHA-256 哈希后得到的 64 个字符的十六进制字符串（只包含 0-9 和 a-f）。完全不包含用户原始密码的任何信息。

思考题3

问题：为什么必须"先压缩后加密"？如果反过来"先加密后压缩"会怎样？

参考回答：加密后的数据近似随机，没有冗余结构，压缩算法几乎无法压缩（压缩率接近 1:1），白白浪费了 CPU 时间。所以正确顺序是压缩 → 加密。

思考题4

问题：代码中 DES 没有显式指定 IV，AES 显式指定了 IV。这种差异会导致什么安全问题？

参考回答：DES 使用全零 IV，如果两份完全相同的明文使用相同密钥加密，密文的前 8 字节会完全相同，攻击者可以据此判断文件内容存在重复模式。AES 从 `hash1` 派生 IV，不同用户即使密码相同，由于 `hash1` 不同（受盐值影响），IV 也不同，相同明文会产生完全不同的密文，更安全。

思考题5

问题：如果备份文件在传输过程中某几个比特发生翻转，CRC64 一定能检测出来吗？

参考回答：不一定。CRC64 的检测能力取决于错误模式和 CRC 多项式。CRC64 能检测出所有 1-4 位的突发错误，以及所有奇数个位的错误，但在极少数情况下（约 2^-64 的概率），错误的文件也可能碰巧通过 CRC 校验。这也是为什么金融级系统会使用更复杂的校验方案（如 SHA-256 + 数字签名）。

思考题6

问题：为什么要用 `extern "C"` 封装 C++ 的加密库？直接用 C++ 类接口不行吗？

参考回答：C++ 的 ABI 在不同编译器之间不兼容，`extern "C"` 确保接口使用 C 的命名和调用约定，任何支持 C 接口的语言（Python、C#、Java 等）都可以调用这个库，提高了代码的复用性和跨语言兼容性。

12. 总结

核心知识点

1. **对称加密**：加密和解密使用相同密钥，适合备份场景
2. **密钥派生**：使用 `SHA256 + 盐值` 将用户密码转换为固定长度密钥
3. **CBC模式**：每个明文块与前一个密文块异或后加密，避免相同明文产生相同密文
4. **PKCS#7补齐**：分组密码要求数据长度是块大小的整数倍
5. **两次哈希策略**：`hash1` 用于派生密钥，`hash2` 用于密码验证
6. **先压缩后加密**：压缩利用冗余，加密消除冗余，顺序不可颠倒
7. **ABI兼容性**：使用 `extern "C"` 包装 C++ 接口，确保跨编译器兼容
8. **CRC64校验**：检测文件完整性，区分意外损坏和正常文件

代码设计亮点

- ✅ 统一的加密算法接口 `IEncryptionAlgorithm`，DES 和 AES 可互换
- ✅ 密钥派生函数 `saltedSHA256` 统一处理密码到密钥的转换
- ✅ OpenSSL EVP API 自动处理补齐，简化实现
- ✅ 文件头设计支持算法标识、CRC校验和密码验证
- ✅ 两次哈希分离密钥派生和密码验证的职责

扩展方向

- 🔧 支持更灵活的密钥管理（如密钥文件代替密码）
- 🔧 支持非对称加密（如将 AES 密钥用 RSA 公钥加密）
- 🔧 支持增量加密（仅加密新增或变更的数据块）
- 🔧 集成硬件安全模块（HSM）或 TPM 进行密钥保护

> 📖 **推荐阅读**:
>
> - OpenSSL 官方文档: https://docs.openssl.org/master/man3/
> - NIST Special Publication 800-38A: Recommendation for Block Cipher Modes of Operation
> - PKCS #7: Cryptographic Message Syntax Standard